MDN has a nice list: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Or, as a short overview:

1×× Informational

- 100 Continue
- 101 Switching Protocols
- 102 Processing

2×× Success

- 200 OK
- 201 Created
- 202 Accepted
- 203 Non-authoritative Information
- 204 No Content
- 205 Reset Content
- 206 Partial Content
- 207 Multi-Status
- 208 Already Reported
- 226 IM Used

3×× Redirection

- 300 Multiple Choices
- 301 Moved Permanently
- 302 Found
- 303 See Other
- 304 Not Modified
- 305 Use Proxy
- 307 Temporary Redirect
- 308 Permanent Redirect

4×× Client Error

- 400 Bad Request
- 401 Unauthorized
- 402 Payment Required

- 403 Forbidden
- 404 Not Found
- 405 Method Not Allowed
- 406 Not Acceptable
- 407 Proxy Authentication Required
- 408 Request Timeout
- 409 Conflict
- 410 Gone
- 411 Length Required
- 412 Precondition Failed
- 413 Payload Too Large
- 414 Request-URI Too Long
- 415 Unsupported Media Type
- 416 Requested Range Not Satisfiable
- 417 Expectation Failed
- 418 I'm a teapot
- 421 Misdirected Request
- 422 Unprocessable Entity
- 423 Locked
- 424 Failed Dependency
- 426 Upgrade Required
- 428 Precondition Required
- 429 Too Many Requests
- 431 Request Header Fields Too Large
- 444 Connection Closed Without Response
- 451 Unavailable For Legal Reasons
- 499 Client Closed Request

5×× Server Error

- 500 Internal Server Error
- 501 Not Implemented
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout
- 505 HTTP Version Not Supported
- 506 Variant Also Negotiates
- 507 Insufficient Storage

- 508 Loop Detected
- 510 Not Extended
- 511 Network Authentication Required
- 599 Network Connect Timeout Error

Source: https://httpstatuses.com/

أولاً: النظرية (The Theory)

في Node.js، الأخطاء بتنقسم لنوعين أساسيين لازم تفرق بينهم زي اسمك:

1. الـ Operational Errors (أخطاء التشغيل)

دي أخطاء "متوقعة" وبتحصل في الظروف العادية، مش عيب في الكود بتاعك بس "الواقع" هو السبب.

- **أمثلة:** الداتابيز وقعت، اليوزر دخل إيميل غلط، فايل مش موجود، الـ Request خد وقت طويل (Timeout).
- **التعامل:** لازم نهندلها بذكاء ونرد على اليوزر برسالة مفهومة.

2. الـ Programmer Errors (أخطاء المبرمج / Bugs)

دي كوارث في الكود نفسه، إنت ناسي حاجة أو كاتب حاجة غلط.

- **أمثلة:** بتحاول تقرأ Property من `undefined` (الشهيرة بـ `Cannot read property of undefined`)، أو ناسي `await` قبل Promise.
- **التعامل:** الحل الوحيد هو إنك تصلح الكود. في العادة، الأفضل إن السيرفر "ينتحر" (Restart) لو حصل النوع ده عشان الداتا متبوظش.

ثانياً: الأنماط الشهيرة للتنفيذ (Popular Patterns)

إليك الطرق اللي "العتاولة" بيستخدموها في الـ Production:

1. الـ Centralized Error Handling Middleware (نمط الوسطجي)

في Express، بدل ما تقعد تعمل `res.status(500)` في كل Controller، إنت بترمي كل الأخطاء لمكان واحد (Centralized).

- **الطريقة:** بتعمل Middleware بياخد 4 Parameters.
- **الكود:**

JavaScript

```
// middleware/error-handler.js
app.use((error, req, res, next) => {
  const status = error.statusCode || 500;
  const message = error.message;
  const data = error.data;
  res.status(status).json({ message: message, data: data });
});
```

- **في الـ Controller:** كل اللي عليك تعمله هو:

JavaScript

```
throw new Error('Something went wrong'); // في الـــ Synchronous
// أو
next(err); // في الـــ Asynchronous
```

2. الـ Async/Await مع Try-Catch (The Modern Way)

بما إن الـ Promises هي أساس النود دلوقتي، الـ `try-catch` بقت هي الملكة.

JavaScript

```
exports.getPost = async (req, res, next) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post) {
      const error = new Error('Could not find post.');
      error.statusCode = 404;
      throw error; // هيروح أوتوماتيك للـــ Catch
    }
    res.status(200).json({ post });
  } catch (err) {
    if (!err.statusCode) err.statusCode = 500;
    next(err); // بيرمي الغلط للميدل وير المركزي
  }
};
```

3. الـ Global "Catch-All" (طوق النجاة الأخير)

ساعات بتحصل أخطاء بره الـ Express خالص (زي غلطة في الـ Database Connection). عشان السيرفر ميقعش فجأة، بنستخدم الـ Events دي في `app.js`:

- **`uncaughtException`**: بتمسك الأخطاء الـ Synchronous اللي ملهاش `try-catch`.

- `unhandledRejection` : بتمسك الـ Promises اللي فشلت ومحدش عمل لها `.catch()` .

JavaScript

```
process.on('unhandledRejection', (reason, promise) => {
  console.log('Unhandled Rejection at:', promise, 'reason:', reason);
  // للغلط قبل ما السيرفر يرستر Log هنا ممكن تبعت إيميل لنفسك أو تعمل
});
```

ثالثاً: الـ Error Object "ليفل الوحش"

ما تكتفيش بـ `new Error('message')` . خليك محترف وزود معلومات للـ Error عشان تساعدك في الـ Debugging:

JavaScript

```
const error = new Error('Validation failed.');
error.statusCode = 422;
error.data = errors.array(); // Array بتاع الـ express-validator
throw error;
```

ملخص لإدارة الأخطاء:

1. **لو Error متوقع (Operational):** استخدم الـ Centralized Middleware وبلغ اليوزر.
2. **لو Error في الـ Async:** لازم تستخدم `next(err)` عشان الـ Express يمسكها.
3. **لو عايز الكود نضيف:** استخدم **Async Wrapper** (مكتبات زي `express-async-handler`) عشان تشيل الـ `try-catch` الكتير من وشك.